Лабораторная работа №4 по дисциплине "Операционные системы, среды и оболочки"

Взаимодействие со стандартными приложениями Windows на VBScript/JScript

Все программы, запускаемые в операционной системе Windows, могут обмениваться информацией, как содержательного характера, так и управляющими сигналами. К экземпляру запущенной программы можно обращаться посредством объекта **`WScript`**, который поддерживает обращение к списку дескрипторов всех запущенных в операционной системе приложений. В результате мы получаем дескриптор процесса и, используя методы объекта **`WScript`**, передаем сообщение программе.

Сообщения, которые можно отправить приложению, указываются в качестве текстового параметра функции **`SendKeys`**. Помимо стандартных символов имеется возможность передавать расширенные команды, вводимые с клавиатуры с использованием функциональных клавиш или клавиш специального назначения (неотображаемые символы такие как [Enter], [Tab], [PageUp] и прочие), для чего их символьное обозначение заключается в фигурные скобки. Некоторые символы в методе **`SendKeys`** имеют специальное назначение **`+ ^ % ~ ( ) { }`**, поэтому при их передаче также необходимо использовать фигурные скобки. В таблице представлены соответствующие значения.

Клавиша	Символьное обозначение	Клавиша	Символьное обозначение
[Backspace]	{BACKSPACE} {BS} {BKSP}	[Page Up]	{PGUP}
[Break]	{BREAK}	[Print Screen]	{PRTSC}
[Caps Lock]	{CAPSLOCK}	[Scroll Lock]	{SCROLLLOCK}
[Delete]	{DELETE} {DEL}	[Tab]	{TAB}
[End]	{END}	[↑]	{UP}
[Enter]	{ENTER}	[←]	{LEFT}
[Home]	{HOME}	[↓]	{DOWN}
[Insert]	{INSERT} {INS}	[→]	{RIGHT}
[Num Lock]	{NUMLOCK}	[F1]	{F1}
[Page Down]	{PGDN}	[Fчисло]	{Fчисло}

Для задания комбинаций клавиш с [Shift], [Ctrl] или [Alt], перед соответствующей комбинацией необходимо указать соответствующий клавише код.

[Shift] **`+`** [Ctrl] **`^`** [Alt] **`%`**

Круглые скобки **`( )`** используются для указания одновременного нажатия нескольких клавиш на клавиатуре. Например, если требуется нажать клавишу [Shift] одновременно с клавишей [G], а потом нажать клавишу [F], то используется следующая текстовая строка для метода **`SendKeys`** **`"+GF"`**. В случае если мы хотим нажать все три клавиши одновременно, необходимо использовать для метода **`SendKeys`** текстовый параметр **`"+(GF)"`**. При имитировании нажатий на экранные клавиши или при использовании «горячих» комбинаций клавиш необходимо делать задержку, чтобы подсистема пользовательского интерфейса успевала отреагировать до поступления новой порции нажатий клавиш. Задержку выполняет метод **`WScript.Sleep(количество миллисекунд)`**. Значением параметра, вполне достаточным для реакции ОС на нажатие клавиш, является 500 миллисекунд.

Для запуска приложения, зарегистрированного в операционной системе необходимо указать его наименование в качестве параметра для метода **`CreateObject`** объекта **`WScript`**. Например, запуск приложения Internet Explorer осуществляется с применением следующей конструкции в тексте программы JScript:

```
var vIE=WScript.CreateObject("InternetExplorer.Application");
```

Аналогичное действие на VBScript будет выглядеть так:

```
Set vIE=WScript.CreateObject("InternetExplorer.Application")
```
После создания экземпляра приложения возможно взаимодействие с переменной-объектом посредством его методов и свойств. Также можно запускать приложения с использованием операционной оболочки. При этом указывается командная строка запуска программы.
`var vShell=WScript.CreateObject("WScript.Shell");` используется для создания переменной-объекта, а метод **Exec** запускает на исполнение командную строку, которая передается в качестве параметра. Возвращаемая методом ссылка на объект процесса, позволяет контролировать его выполнение, так как управление сразу же передается сценарию. `var vProcess=vShell.Exec("программа");`. Для проверки хода работы запущенной задачи используется свойство объекта **Status**, которое принимает значение 1 в случае завершения процесса, и 0 – когда работа еще не завершилась. Например, можно ожидать в цикле завершение выполнения программы `while (vProcess.Status==0) WScript.Sleep(100);`. По окончанию работы, процесс сохраняет в свойстве **ExitCode** код завершения. Исполнение командной строки происходит в консольном режиме, поэтому доступными для сценария становятся объекты **StdIn**, **StdOut** переменной-экземпляра запущенного процесса. При запуске команд операционной системы MS-DOS необходимо вначале параметра указывать интерпретатор командной строки, путь к которому содержится в переменной среды окружения **%COMSPEC%**. К примеру, запуск команды MS-DOS на считывание списка содержимого системной директории выглядит так

```
var vShell=WScript.CreateObject("WScript.Shell");
var vProcess=vShell.Exec("%COMSPEC% /C DIR %SystemRoot% /W");
while (!vProcess.StdOut.AtEndOfStream){
WScript.Echo(vProcess.StdOut.ReadLine());
}
```

Свойство **vProcess.ProcessID** позволяет получить числовое значение дескриптора процесса, которое может быть использовано при переключении между несколькими запущенными задачами.
Еще одним способом запуска программ из скриптового файла является использование метода **Run** объекта **WScript.Shell**. Например, для запуска приложения «Калькулятор» необходимо в программе записать строчку `vShell.Run("calc.exe",1)`. Второй параметр указывает способ отображения окна программы и может принимать такие значения как

	Константа VBScript	Описание
0	vbHide	Прячет текущее окно, фокус ввода передается следующему в диспетчере задач приложению.
1	vbNormalFocus	Активизирует и отображает окно. Ели окно было минимизировано или максимизировано, система восстановит его первоначальное положение и размер. Обычно указывается в качестве параметра для первого запуска программы.
2	vbMinimizedFocus	Активизирует окно и отображает его в минимизированном (свернутом в панели задач) виде.
3	vbMaximizedFocus	Активизирует окно и отображает его в максимизированном (развернутом во весь экран) виде.
4	vbNormalNoFocus	Отображает окно в том виде, в котором оно находилось последний раз. Фокус остается за предыдущим активным окном.
5		Активизирует окно и отображает его в текущем состоянии.
6	vbMinimizedNoFocus	Минимизирует заданное окно и активизирует следующее, расположенное за ним, окно.
7		Отображает окно в свернутом виде. Активное окно при этом остается активным.
8		Отображает окно в его текущем состоянии. Активное окно при

		этом остается активным.
10		Устанавливает режим отображение, такой же, как и у программы, запускающей данное приложение.

В сценариях, написанных на VBScript, можно непосредственно использовать именованные константы, например **vbHide**, без предварительного их объявления. Для того чтобы использовать такие константы в JScript, их необходимо объявить как переменные и присвоить соответствующие значения, например `var vbHide=0;`.
Третий параметр для метода **Run** является необязательным, и передает значение логического типа **true** или **false**. При истинном значении сценарий ожидает завершения выполнения запущенного приложения, а по окончании получает от метода **Run** код завершения программы. По умолчанию, когда третий параметр не указывается, метод **Run** сразу же возвращает управление сценарию, что соответствует значению **false**.
Переключение между несколькими запущенными процессами осуществляется при помощи метода **vShell.AppActivate**, который принимает в качестве параметра дескриптор задачи или заголовок окошка, в котором выполняется приложение. Во втором случае, для того чтобы определить, какое именно приложение необходимо активизировать, текстовая строка параметра сравнивается по очереди с названиями окон всех запущенных приложений. Если не найдено ни одного точного совпадения, то будет произведено сопоставление с тем окном, название которого начинается со значения текстовой строки параметра. Если и это не приведет к идентификации программы, то сопоставление будет произведено с наименованиями окон запущенных программ, оканчивающихся на текстовую строку параметра. Если найдено несколько похожих по наименованию окон, активизируется одно из них в произвольном порядке.

Создайте каталог **Lab4** в директории **C:\STUDENT\группа**. Запишите программу **script1.vbs**, которая будет выполнять следующие действия: запускает приложение «Калькулятор» и «Блокнот», вычисляет пример «9*6=» с использованием калькулятора, результат вычислений помещает в блокнот.
Запишите программу **script2.vbs**, которая будет выполнять следующие действия: запускает приложение «Калькулятор» и «Блокнот», вычисляет пример «F7+7F=» с использованием калькулятора, результат вычислений помещает в блокнот в шестнадцатеричном виде, а потом дополнительной строкой в десятичном. Строчки в блокноте должны начинаться со слов, поясняющих выводимый результат, то есть

```
Шестнадцатеричный результат: число из калькулятора
Десятичный результат: число из калькулятора
```

По окончанию вычислений программа «Калькулятор» должна приобрести вид «Обычный».

При работе с программами, иногда возникает необходимость прочитать или записать определенную информацию, с использованием файлов. Для работы в файловой системе, необходимо получить доступ к ее функциям. Такой доступ предоставляет объект **Scripting.FileSystemObject**. Создание экземпляра объекта и присвоение ссылки на объект переменной осуществляется через метод **CreateObject**.

```
var FSO=WScript.CreateObject("Scripting.FileSystemObject");
```

После получения ссылки на экземпляр объекта, сценарий может осуществлять операции файлового ввода/вывода, и также операции связанные с каталогами. Вот перечень основных операций:
CopyFile("путь источника","путь расположения"[,overwrite]) копирует один или несколько файлов из одного места в другое, необязательный параметр overwrite логического типа указывает на то, необходимо ли перезаписывать существующие в каталоге назначения файлы.

CopyFolder("путь источника","путь расположения"[,overwrite]) копирует каталог со всеми подкаталогами из одного места в друге, необязательный параметр overwrite логического типа определяет необходимость перезаписи существующих файлов и каталогов.

CreateTextFile("путь к создаваемому файлу или просто имя файла"[,overwrite]) создает текстовый файл и возвращает указывающий на него объект-ссылку. Необязательный параметр overwrite логического типа указывает на необходимость перезаписи существующего файла, информация в котором будет утеряна при перезаписи новыми значениями.

OpenTextFile("путь к создаваемому файлу или просто имя файла"[,режим[,create]]) открывает текстовый файл и возвращает указывающий на него объект-ссылку, в зависимости от значения необязательного параметра create логического типа отсутствующий по указанному пути файл может быть создан. При открытии файла указывается режим его использования: 1 – только для чтения, 2 – для записи, 8 – для дополнения.

CreateFolder("путь к создаваемому каталогу или просто имя каталога") создает новый каталог с указанным в качестве параметра именем.

DeleteFile("путь к удаляемому файлу или просто имя файла") удаляет указанный в качестве параметра файл.

DeleteFolder("путь к удаляемому каталогу или просто имя каталога") удаляет указанный в качестве параметра каталог со всем его содержимым.

FileExist("путь к файлу или просто имя файла") возвращает логическое значение в зависимости от того существует ли по указанному в качестве параметра пути файл.

FolderExist("путь к каталогу или просто имя каталога") возвращает логическое значение в зависимости от того существует ли каталог по указанному пути.

При работе с созданным или открытым файлом все операции ввода/вывода производятся через объект-ссылку. Методы и свойства, которые при этом понадобятся перечислены ниже.

AtEndOfLine логическое свойство доступное только для чтения, отображает факт достижения окончания строки (нажатия клавиши [Enter]) указателем внутри файла.

AtEndOfStream логическое свойство доступное только для чтения, отображает факт достижения окончания файла.

Close() закрывает открытый или созданный файл.

Read(n) считывает из файла n символов.

ReadAll() считывает полностью весь файл.

ReadLine() возвращает считанную из файла строку.

Skip(n) пропускает при чтении n символов.

SkipLine() пропускает при чтении целую строку.

Write(text) выводит в файл текстовое сообщение text.

WriteLine([text]) выводит в файл необязательный для указания параметр сообщения text и завершает его переводом строки.

WriteBlankLines(n) добавляет в файл несколько пустых строк в количестве n.

Для взаимодействия с пользователем сценарий может выдавать диалоговые окошки стандартного вида с некоторым набором кнопок и ожидать ответа от пользователя. Метод Popup объекта WScript.Shell выводит такой диалог. Сценарий на момент вывода диалога останавливается и ожидает ответа пользователя (при указании тайм-аута идет отсчет времени ответа). Возвращаемое методом Popup значение указывает вариант выбора пользователя. Формат метода следующий **Popup("текст сообщения"[,количество секунд ожидания ответа][,"текст заголовка"][,определяющий количество кнопок и вид значка тип диалога])**. Тип диалога – это число,

которое получается сложением значения соответствующего набора кнопок и соответствующего значка-изображения в диалоге.

	Константа VBScript	Описание
0	vbOkOnly	Выводится кнопка OK.
1	vbOkCancel	Выводятся.кнопки OK и Cancel.
2	vbAbortRetryIgnore	Выводятся кнопки Abort, Retry и Ignore.
3	vbYesNoCancel	Выводятся кнопки Yes, No и Cancel.
4	vbYesNo	Выводятся кнопки Yes и No.
5	vbRetryCancel	Выводятся кнопки Retry и Cancel.
16	vbCritical	Выводится значок Stop.
32	vbQuestion	Выводится значок Question.
48	vbExclamation	Выводится значок Exclamation.
64	vbInformation	Выводится значок Information.

Варианты возвращаемых значений выбора, сделанного пользователем, следующие

	Константа VBScript	Описание
-1		Пользователь не нажал ни одну кнопку в течении времени, заданного параметром тайм-аута.
1	vbOK	Нажата кнопка OK..
2	vbCancel	Нажата кнопка Cancel.
3	vbAbort	Нажата кнопка Abort.
4	vbRetry	Нажата кнопка Retry.
5	vbIgnore	Нажата кнопка Ignore.
6	vbYes	Нажата кнопка Yes.
7	vbNo	Нажата кнопка No.

Использование метода Popup делает выполнение сценария скрипта интерактивным, и позволяет организовать сложные диалоговые опросы о действиях, которые намерен совершить пользователь.

Создайте программу **`script3.js`** которая будет выполнять вычисления посредством приложения «Калькулятор», считывая построчно примеры из заранее подготовленного текстового файла **`text1.txt`**. Результаты вычислений программа должна помещать в текстовый файл **`text2.txt`** в порядке обработки примеров из файла **`text1.txt`**. Форматы обозначения чисел в системах счисления должны соответствовать обозначениям префикса перед значащими цифрами: 0хЧИСЛО – шестнадцатеричное, ЧИСЛО – десятичное, 0ЧИСЛО – восьмеричное, @ЧИСЛО – двоичное. Числа могут быть дробными, разделяющим целую и дробную части символом является точка. Арифметические операции в примерах предполагают стандартные действия + - * /. Приоритет арифметических операций может быть указан с помощью круглых скобок, что должно учитываться программой. Результаты вычислений помещаются в заданной пользователем системе счисления (шестнадцатеричная, десятичная, восьмеричная, двоичная), о чем программа должна спросить пользователя при помощи диалога в момент запуска. Если число задано отдельно без арифметической операции, то следует преобразовать его к виду, соответственно указанной пользователем системы счисления.